

これからのコンテンツポータルとして、外部も投稿できるプラットフォームに昇格させたいんだけど、構想を練って、実装できるところまで落とし込むのをてつだってもらってもいい？

 ChatGPT gpt-5-5-thinking

もちろん。これはかなり面白いです。いまの「社内ポータル」をそのまま広げるより、**社内Hub → 審査制投稿 → 販売/配布プラットフォーム**の順で育てる設計にした方が事故りにくいです。まずは構想を「ビジコンで語れる未来」と「Codexに投げられる実装仕様」の両方に分解します。

もちろん。

これはもう、かなり本気で育てる価値あると思います。

今の **LLLD Works Hub** は、

社内で使う便利ツール置き場

だけど、外部投稿型に昇格させるなら、目指す姿はこうです。

構想の芯

「現場で生まれた小さな便利ツールを、誰でも投稿・販売・利用できる場所」

たとえば今ある、

- 勤怠アプリ「だこくん」
- 座席管理アプリ
- PDF編集ツール
- 小テスト作成ツール
- 面談支援ツール
- 補助金テンプレート
- 教室運営資料
- コンサル支援資料

みたいなものを、LLLDだけが出すのではなく、

塾長、店長、事務員、講師、現場スタッフ、個人事業主が
「うちで作った便利なもの」を投稿できるマーケット

にする感じです。

かなり刺さる表現にすると、

現場の困りごとを、現場の人が解決し、他の現場にも届けられるプラットフォーム

です。

いきなり全部作らない方がいい

最初からログイン、決済、投稿審査、売上分配、ユーザー管理まで作ると重すぎます。だから段階はこうです。

Phase 1: 社内ポータル改造版

まずは今の GitHub Pages のまま、外部投稿っぽい見た目にする。

機能はまだシンプルでOK。

- トップページを「マーケット風」に変更
- 投稿コンテンツ一覧カード
- カテゴリ検索
- タグ検索
- 無料/有料/社内限定の表示
- 投稿者カード
- コンテンツ詳細ページ
- 「使ってみる」「資料を見る」「申請する」ボタン
- 投稿募集ページ
- Googleフォーム等で投稿受付

この段階では、実際の投稿機能はまだフォームで代用。でも見た目は完全にプラットフォームに寄せる。

ここがビジコン・営業・社内説明に一番使いやすいです。

Phase 2: 審査制コンテンツ投稿

次に、外部の人が投稿できる仕組みを作る。

ただし、最初は完全自動投稿ではなく、**審査制**がいいです。

理由はシンプルで、

- 変なファイルが投稿される

- 著作権的に危ないものが混ざる
- 動かないHTMLやアプリが投稿される
- 品質バラつく
- 料金トラブルが起きる

このリスクを避けるため。

投稿フローはこう。

1. 投稿者がフォームから申請
2. LLLD側で内容確認
3. サムネイル・説明文・タグを整える
4. Works Hub に掲載
5. 無料配布、問い合わせ、有料販売へ誘導

最初はこれで十分です。

Phase 3: 本当のマーケットプレイス化

ここで初めて、

- 会員登録
- 投稿者マイページ
- 購入者マイページ
- 決済
- ダウンロード管理
- レビュー
- 売上分配
- 月額利用
- 人気ランキング
- おすすめ表示
- 広告枠

を入れる。

ここまで来たら、GitHub Pages単体では厳しいので、Supabase / Firebase / Stripeなどを組み合わせる形になると思います。

最初に作るべきMVP

最初のMVP名は仮で、

LLLD Works Market

または、

現場ツールマーケット

くらいがわかりやすいです。

最初に実装する画面はこの8つ。

1. トップページ

目的は、プラットフォーム感を出すこと。

必要要素：

- ヒーローコピー
- 人気コンテンツ
- 新着コンテンツ
- カテゴリー一覧
- 投稿者募集
- LLLD公式コンテンツ
- 外部投稿コンテンツ
- 無料で使えるコンテンツ
- 有料/準備中コンテンツ

コピー案：

現場で生まれた便利ツールを、必要な人へ。

教室・店舗・事務所の小さな困りごとを解決する、実務コンテンツマーケット。

2. コンテンツ一覧ページ

カード型で並べる。

カード項目：

- サムネイル
- タイトル
- カテゴリー

- 対象業種
- 無料/有料/社内限定
- 投稿者
- 更新日
- ひとこと説明
- CTA

例:

だこくん
小規模事業者向け勤怠管理ツール
対象: 塾・店舗・小規模事務所
価格: 無料β版
投稿者: LLLD公式

3. コンテンツ詳細ページ

ここが販売ページになります。

必要項目:

- 何ができるか
- 誰向けか
- 解決する悩み
- 画面キャプチャ
- 主な機能
- 使い方
- 注意事項
- 更新履歴
- 投稿者情報
- ダウンロード/利用ボタン
- 問い合わせボタン

詳細ページはかなり重要です。

ここがLPっぽくなると、販売力が出ます。

4. 投稿者募集ページ

ここはビジコン的にも大事。

コピー案:

あなたの現場で使っている便利な資料・ツール・テンプレートを、他の現場にも届けませんか？

投稿対象:

- 業務テンプレート
- Excel管理表
- HTMLアプリ
- 授業資料
- 面談資料
- チェックリスト
- マニュアル
- 小規模事業向けツール

掲載条件:

- 実際に現場で使ったことがある
- 他社でも使える形になっている
- 著作権的に問題がない
- 個人情報を含まない
- 説明文・サムネイル作成に協力できる

5. カテゴリページ

最初はこのカテゴリでいいです。

- 教育・塾運営
- 店舗運営
- 勤怠・人事
- 業務効率化
- AI活用
- PDF・資料編集
- 面談・営業支援
- 補助金・行政書類
- テンプレート

- 社内限定

6. 投稿者ページ

投稿者の信頼を見せる場所。

項目：

- 投稿者名
- 所属/屋号
- 得意分野
- 投稿コンテンツ一覧
- 実績
- コメント
- 問い合わせボタン

将来的に「人気投稿者」が生まれると強いです。

7. 管理用データファイル

GitHub Pagesで最初の実装するなら、投稿データは `JSON` で持つのがよさそうです。

例：

```
json

{
  "id": "dakokun",
  "title": "だこくん",
  "category": "勤怠・人事",
  "target": ["塾", "小規模事業者", "店舗"],
  "priceType": "無料β版",
  "author": "LLLD公式",
  "updatedAt": "2026-06-24",
  "thumbnail": "./assets/thumbs/dakokun.png",
  "summary": "小規模事業者向けのシンプルな勤怠管理ツール。",
  "status": "公開中",
  "tags": ["勤怠", "QR", "Excel出力", "小規模事業"],
  "url": "./contents/dakokun/index.html"
}
```

これにしておけば、後から外部投稿が増えても管理しやすいです。

8. 投稿受付フォーム

最初はGoogleフォームでOK。

フォーム項目：

- 投稿者名
- メールアドレス
- コンテンツ名
- コンテンツ種別
- 対象業種
- 解決する悩み
- 使い方
- 希望価格
- 無料公開可否
- 添付ファイル
- サムネイル素材
- 掲載希望コメント

実装方針

今の技術方針なら、最初はこれが一番良いです。

txt

```
l1ld-works-hub/  
├── index.html  
├── marketplace.html  
├── submit.html  
├── author.html  
├── content-detail.html  
├── assets/  
│   ├── css/  
│   │   └── style.css  
│   ├── js/  
│   │   ├── app.js  
│   │   ├── marketplace.js  
│   │   └── content-data.js  
│   ├── thumbs/  
│   └── images/  
└── data/
```



```
| |—— contents.json
| |—— categories.json
| |—— authors.json
|—— contents/
| |—— dakokun/
| |—— seatflow/
| |—— pdf-editor/
| |—— interview-support/
|—— AGENTS.md
```

GitHub Pages運用を考えるなら、まずはこれで十分です。

ビジコンでの見せ方

この構想は、単なる「コンテンツ販売サイト」と言うと弱いですが、
でも、こう言うと強くなります。

AI時代、現場の人でも小さな業務アプリやテンプレートを作れるようになりました。
しかし、それを販売・共有・改善する場所がありません。
私たちは、現場で生まれた小さな解決策を、他の現場に届けるプラットフォームを作ります。

さらに、

プログラマーだけがソフトを作る時代から、現場の人が自分の業務を改善し、その成果を販売
できる時代へ。

これはかなり良いです。

まず最初にやること

最初の一手はこれです。

現在のLLLD Works Hubを「マーケットプレイス風トップ」に改造する

具体的には、

- 社内ポータル感を少し弱める
- カードを「商品カード」に寄せる
- 投稿者情報を追加
- 無料/有料/社内限定ラベルを追加
- カテゴリ検索を強化
- 投稿募集導線を追加
- 詳細ページをLP風にする

- データ管理をJSON化する

これをやれば、ビジコン資料にも、社内運用にも、外部説明にも使えるプロトタイプになります。

Codexに投げるなら最初の指示はこれ

md

LLLD Works Hub 外部投稿型プラットフォーム化 v0.1

目的

現在のLLLD Works Hubを、社内ポータルから将来的に外部投稿型コンテンツマーケットへ拡張できる構成に変更してください。

ただし、現時点ではログイン・決済・自動投稿機能は実装しません。
GitHub Pagesで安定運用できる静的サイトとして実装してください。

方向性

「社内便利ツール置き場」ではなく、
「現場で生まれた便利ツール・資料・テンプレートを探せるマーケット」に見えるUI/UXへ変更してください。

必須実装

- トップページをマーケットプレイス風に再設計
- コンテンツカードに以下を追加
 - サムネイル
 - カテゴリ
 - 投稿者
 - 無料/有料/社内限定/準備中のラベル
 - 更新日
 - 対象業種
 - タグ
 - CTAボタン
- 最終利用履歴8件は維持
- カテゴリ検索・タグ検索を追加
- 投稿者募集セクションを追加
- 投稿受付ページ submit.html を追加
- コンテンツ詳細ページのテンプレートを追加
- data/contents.json でコンテンツ情報を管理
- data/authors.json で投稿者情報を管理
- data/categories.json でカテゴリ情報を管理

デザイン方針

- 日本人向けのLP/業務ツールマーケットの見た目
- 余白は広め
- カードは視認性重視
- 画像・サムネイルで内容がわかる

- 小規模事業者・塾・店舗運営者が迷わない文言
- 不要な説明文は増やさない
- 社内運用にも外部説明にも使える落ち着いたデザイン

実装制約

- GitHub Pagesで動くこと
- 外部ライブラリ依存を増やしすぎない
- APIやサーバーは使わない
- 投稿機能はGoogleフォーム等へのリンクで代替
- 将来SupabaseやFirebaseに移行しやすいデータ構成にする

追加してほしいページ

- marketplace.html
- submit.html
- content-detail.html
- author.html

注意

現在のLLLD Works Hubの良さである、
「よく使うものにすぐアクセスできる」
「社内ツールとして迷わない」
という特徴は消さないでください。

社内用と外部公開用の中間段階として、
“マーケット化された社内ポータル” を作ってください。

まずはこの方向でいくのが一番きれいです。

最初から本物の販売機能まで作るより、**見た目・構造・掲載思想をマーケットに変える**。ここから始めるのが勝ち筋です。

2026-06-24 18:28:12 You Asked



今github でやってるけど、これは社内でも動いているので、動作・仕様確認のために、ローカルで動く仕様にしたいかな。

2026-06-24 18:28:38 You Asked



今github でやってるけど、これは社内でも動いているので、動作・仕様確認のために、ローカルで動かすためのコピーを1つ作成して、動かして、実装したいかな

いいですね、それが一番安全です。**本番GitHub Pagesは社内運用のまま維持して**、別に「ローカル検証用コピー」を作って、そこでマーケット化の実装を進めるのが事故らないです。

うん、方針としてはこれがベストです。

やりたい形

今の本番はそのまま。

txt

GitHub Pages本番
<https://projectllld-blip.github.io/llld-works-hub/>

これは社内で動いているので触らない。

そのうえで、PCの中に

txt

llld-works-hub-local

みたいな**検証用コピー**を作って、そこで

- 外部投稿型プラットフォーム化
- UI変更
- JSONデータ化
- コンテンツ詳細ページ追加
- 投稿者ページ追加
- 投稿フォーム導線追加

を試す。

動作確認して問題なければ、GitHub本番に反映する流れがいいです。

おすすめ運用

本番

txt

```
l1ld-works-hub
```

社内で使っている現行版。
ここは安定運用。

ローカル検証版

```
txt
```

```
l1ld-works-hub-local
```

マーケット化・外部投稿化を試す場所。
壊れてもOK。

ローカルで動かす基本手順

Macならこれでいけます。

```
bash
```

```
cd ~/Desktop
git clone https://github.com/projectl1ld-blip/l1ld-works-hub.git l1ld-works-hub-local
cd l1ld-works-hub-local
python3 -m http.server 5500
```

そのあとブラウザで開く。

```
txt
```

```
http://localhost:5500/
```

ポイントは、**HTMLファイルをダブルクリックで開かないこと**です。
JSON読み込みや画像パスの確認をするなら、必ず `localhost` で動かした方がいいです。

実装の進め方

流れはこうですね。

```
txt
```

1. GitHub上の現行版をローカルにコピー
2. localhostで起動
3. 現行版がローカルでも正しく動くか確認
4. market化用のファイル構成に変更

5. UI・JSON・詳細ページを実装
6. ローカルで動作確認
7. 問題なければGitHubへ反映
8. GitHub Pagesで最終確認

ローカルコピーで最初に追加する構成

まずはこうしたいです。

txt

```
l1ld-works-hub-local/  
├── index.html  
├── marketplace.html  
├── submit.html  
├── content-detail.html  
├── author.html  
├── assets/  
│   ├── css/  
│   │   └── style.css  
│   ├── js/  
│   │   ├── app.js  
│   │   ├── marketplace.js  
│   │   └── content-detail.js  
│   ├── thumbs/  
│   └── images/  
├── data/  
│   ├── contents.json  
│   ├── authors.json  
│   └── categories.json  
├── contents/  
│   ├── dakokun/  
│   ├── seatflow/  
│   ├── pdf-editor/  
│   └── interview-support/  
└── AGENTS.md
```

特に大事なものは `data/contents.json` です。

ここにコンテンツ情報をまとめておけば、今後投稿コンテンツが増えてもHTMLを毎回ベタ書きしなくてよくなります。

Codexに投げる指示

そのまま投げるなら、これでいいです。

LLLD Works Hub ローカル検証版の作成とマーケット化 v0.1

目的

現在GitHub Pagesで社内運用している LLLD Works Hub を壊さずに、ローカル検証用コピーとして動かせる状態にしてください。

このローカル版を使って、将来的に外部投稿型コンテンツプラットフォームへ拡張できる実装を進めます。

前提

- 現在のGitHub Pages本番は社内では使っているため、直接壊さないこと
- ローカル環境で `python3 -m http.server 5500` などを使って動作確認できるようにすること
- HTMLファイルの直接ダブルクリックではなく、localhostで確認する前提にすること
- GitHub Pagesでも動く静的サイト構成を維持すること
- API、ログイン、決済、自動投稿はまだ実装しないこと

最初に行うこと

1. 現行構成を確認する
2. ローカルで `http://localhost:5500/` から正しく表示されるか確認する
3. パス指定がGitHub Pages専用になっていてローカル表示で崩れる箇所があれば、相対パス中心に修正する
4. 既存の社内運用導線は壊さない
5. 新しいマーケット化用のページとデータ構成を追加する

追加したい構成

以下を追加・整理してください。

- marketplace.html
- submit.html
- content-detail.html
- author.html
- data/contents.json
- data/authors.json
- data/categories.json
- assets/js/marketplace.js
- assets/js/content-detail.js
- assets/css/style.css の整理
- assets/thumbs/ サムネイル格納場所

実装方針

現在のLLLD Works Hubを、外部投稿型マーケットプレイスへ拡張しやすい構成にしてください。

ただし、現時点では「マーケット化された社内ポータル」として実装してください。

必須要件

- 社内によく使うコンテンツへのアクセス性は維持する
- 最終利用履歴8件の思想は維持する

- コンテンツカードに以下の情報を表示する
 - サムネイル
 - タイトル
 - カテゴリ
 - 投稿者
 - 無料 / 有料 / 社内限定 / 準備中 ラベル
 - 更新日
 - 対象業種
 - タグ
 - CTAボタン
- コンテンツ情報はHTML直書きではなく、基本的に `data/contents.json` から読み込む
- 投稿者情報は `data/authors.json` に分離する
- カテゴリ情報は `data/categories.json` に分離する
- ローカル環境とGitHub Pagesの両方で動くように、パス設計に注意する

デザイン方針

- 日本の小規模事業者・塾・店舗運営者に伝わるUI
- SaaSっぽすぎず、現場の人が迷わない
- 商品カードはサムネイル重視
- 説明文は短く、見た瞬間に用途がわかる
- 「社内ツール置き場」ではなく「現場コンテンツマーケット」に見えること
- ただし、社内利用者が迷うほど構造を変えすぎない

ローカル確認

以下のコマンドで動くことを確認してください。

```
```bash
python3 -m http.server 5500
```

確認URL:

txt

<http://localhost:5500/>

## 注意

- 本番公開中のファイルを直接危険な形で壊さない
- 既存コンテンツへのリンク切れを起こさない
- 相対パスを優先する
- 不要な外部ライブラリを増やさない
- まずは静的HTML / CSS / JS / JSON で完結させる

text



---

# たぶん最初に詰まるポイント

一番ありそうなのは、\*\*パス問題\*\*です。

GitHub Pagesだと、

```
```txt
/llld-works-hub/
```

配下で動くけど、ローカルだと

txt

<http://localhost:5500/>

で動く。

この違いで、画像・CSS・JS・JSONが読み込めないことがあります。

なので、実装ではできるだけこうします。

html

```
<link rel="stylesheet" href="./assets/css/style.css">
<script src="./assets/js/app.js"></script>
```

こういう絶対パスは避けたいです。

html

```
<link rel="stylesheet" href="/llld-works-hub/assets/css/style.css">
```

かなり安全な進め方

個人的には、この順番がいいです。

Step 1

現行版をローカルコピーして、まず何も変更せずに動かす。

Step 2

動いたら、マーケット化用のページだけ追加する。

txt

```
marketplace.html
submit.html
content-detail.html
author.html
```

Step 3

既存トップはすぐ全面改修せず、まずはトップに

txt

外部投稿型マーケット β

への導線を追加する。

Step 4

マーケットページが完成してから、トップページを徐々に寄せる。

これなら社内利用者が急に迷子にならないです。

まずの完成イメージ

最初のローカル版では、これを目標にするといいです。

txt

LLLD Works Hub Local β

- 社内でよく使うもの
- 最終利用履歴8件
- LLLD公式コンテンツ
- 外部投稿予定コンテンツ
- 無料β版
- 有料予定
- 投稿者募集
- コンテンツ詳細ページ
- 投稿申請フォーム導線

この状態まで作れたら、かなり良いです。

本番を壊さず、ビジコンにも見せられて、今後の販売プラットフォーム化にもつながります。

あとね、アカウントで管理するようになるから、ローカルにデータが保存されるHTML形式は使えなくなるんだよね。そこについてはどうすればいいと思う？

 ChatGPT gpt-5-5-thinking

そこ、めちゃくちゃ重要です。結論から言うと、**HTML単体アプリ路線から「フロントHTML+クラウドDB+認証」路線へ段階移行**するのが現実的です。まず選択肢を整理して、LLLD向けの落としどころまで絞ります。

うん、その認識は正しいです。

アカウント管理するなら、localStorage前提のHTMLアプリは限界が来ます。

ただし、全部を捨てる必要はないです。
整理するとうです。

結論

今後は、

HTML単体アプリ

から

フロントHTML + クラウドDB + 認証 + ストレージ

に移行するのがいいです。

GitHub Pagesはそのまま使えます。

ただし、役割を変えます。

txt

GitHub Pages

= 見た目・画面・フロントエンドを置く場所

Supabase / Firebase など

= アカウント・投稿データ・購入履歴・保存データを管理する場所

つまり、GitHub Pagesは「サイトの外側」として残し、データ管理だけクラウド側に逃がす感じですよ。

まず理解しておくべきこと

今のHTMLアプリでよく使っている保存方法は、たぶんこういうやつです。

txt

localStorage
IndexedDB
ブラウザ内保存
JSONダウンロード
Excelエクスポート

これは社内ツールならめっちゃくちゃ便利です。
でも、外部投稿型プラットフォームにすると弱いです。

理由は、

- 端末ごとにデータが分かれる
- ログインしても別PCでは続きが見えない
- 購入履歴を管理できない
- 投稿者ごとの管理ができない
- 管理者が審査できない
- 売上や公開状態を管理できない
- ユーザー権限を制御できない
- データ消失リスクがある

からです。

なので、**ポータル本体の情報はローカル保存しない方向に変えるべきです。**

ただし、全部クラウド保存にしないでいい

ここが大事です。

クラウド保存すべきもの

これは必ずクラウド側。

txt

- ユーザーアカウント
- 投稿者情報
- コンテンツ一覧
- コンテンツ詳細
- 公開/非公開ステータス
- 審査状況

- 購入履歴
- お気に入り
- 利用権限
- レビュー
- 売上管理
- 管理者メモ
- 組織アカウント

ローカル保存でもよいもの

これはローカルでも残していいです。

txt

- PDF編集の一時データ
- 画像変換の一時データ
- 画面の表示設定
- 最後に開いたタブ
- 入力途中の一時下書き
- ログイン不要の無料ツールの作業データ

つまり、

マーケットの管理データはクラウド
作業中の一時データはローカルでもOK

という分け方がよさそうです。

LLLDにおすすめの構成

個人的には、最初は **Supabase** が一番合っていると思います。

理由は、

- アカウント認証がある
- データベースがPostgres
- ファイル保存もできる
- 権限管理がしやすい
- GitHub Pagesなどの静的フロントから使いやすい
- 将来、本格サービス化しても拡張しやすい

SupabaseはAuth、Postgresデータベース、Storage、Realtime、Edge Functionsなどをまとめて提供していて、ブラウザからDBを使う場合もRow Level Securityでユーザーごとのアクセス制御を設計できます。([Supabase](#))

Firestoreでもできる

Firestoreでももちろん可能です。

Firestore Authenticationはメール/パスワードログインやパスワードリセットなどを扱えますし、Cloud FirestoreはWebアプリ向けのスケーラブルなデータベースとして使えます。(Firestore)

ただ、LLLDの今後を考えると、

- 投稿審査
- 売上管理
- コンテンツ権限
- 組織アカウント
- 将来的な管理画面
- データ分析

が出てくるので、個人的には **Supabaseの方が管理しやすそう**です。

Firestoreはアプリっぽいリアルタイム体験には強い。

Supabaseは業務管理・権限管理・販売管理っぽい構造に向いている。

そんなイメージです。

Cloudflare構成もあり

もう少し開発者寄りにするなら、

```
txt
```

```
Cloudflare Pages
Cloudflare Functions
D1 Database
R2 Storage
```

でも作れます。

Cloudflare Pages Functionsでは認証、フォーム送信、ミドルウェアなどのアプリ的な処理を追加でき、Pages FunctionsからD1などのリソースを使うBindingも用意されています。(Cloudflare Docs)

ただ、最初のLLLD Works Marketではちょっと開発色が強いです。

Codexで作るには悪くないけど、管理・運用のしやすさではSupabaseの方が入りやすいと思います。

決済まで考えるなら

将来的に販売プラットフォームにするなら、決済はStripe Connectが候補です。

Stripe Connectはマーケットプレイスやプラットフォームで、複数の販売者に対する決済・送金管理をするための仕組みです。(Stripe Docs)

ただし、これは最初から入れない方がいいです。

最初は、

txt

無料公開
問い合わせ販売
手動請求
銀行振込
Stripeの単発決済リンク

くらいで十分です。

本当に投稿者が増えて、売上分配が必要になったらStripe Connectに進めばいいです。

じゃあ今どうすべきか

おすすめはこの段階移行です。

Phase 0: ローカル検証用コピー

まだDBは使わず、JSONでマーケットの見た目を作る。

txt

data/contents.json
data/authors.json
data/categories.json

これはUI確認用。

本番データではなく、モックデータ。

目的は、

txt

このプラットフォームはどう見えるか？
カード設計はわかりやすいか？
投稿者ページは必要か？
詳細ページは販売できそうか？

を見ること。

Phase 1: Supabase検証環境を作る

ここからアカウント制にする。

txt

Supabase Staging
= 検証用DB

本番DBではなく、検証用プロジェクトを1つ作るのが安全です。

ここで実装するもの。

txt

- ログイン
- ユーザー登録
- 投稿者プロフィール
- コンテンツ一覧
- コンテンツ詳細
- お気に入り
- 投稿申請
- 管理者だけが公開できる仕組み

Phase 2: 社内β版

外部公開ではなく、まず社内・関係者だけで試す。

txt

- LLLD公式コンテンツ
- 講師作成コンテンツ
- 知人事業者のサンプル投稿
- 無料公開
- 問い合わせ導線

ここではまだ決済なしでOK。

Phase 3: 販売機能

ここで初めて、

txt

- 購入履歴
- ダウンロード権限
- 有料コンテンツ
- 売上管理
- Stripe連携

を考える。

重要な設計変更

今後は、コンテンツをこう分けた方がいいです。

1. ポータル管理データ

これはクラウドDB。

txt

contents
authors
categories
users
organizations
purchases
favorites
reviews
submissions

2. コンテンツ本体

これは種類によって分ける。

txt

HTMLアプリ
PDF
Excel
Word
画像
ZIP
Google Driveリンク
外部URL

3. ユーザーの作業データ

これは方針を分ける。

txt

無料ツールの一時作業
→ localStorageでもOK

ログインユーザーの保存データ
→ Supabase DB / Storageへ保存

個人情報を含むデータ
→ 原則クラウド保存しない、または厳密に権限管理

ここめちゃくちゃ大事です。

たとえばPDF編集ツールなら、PDFの中身を勝手にクラウド保存しない方がいいです。
「端末内で処理します」と言える方が安心感があります。

一方で、購入履歴やお気に入りにはクラウド保存しないと意味がないです。

具体的なDBイメージ

Supabaseにするなら、最初のテーブルはこれくらい。

txt

```
profiles
- id
- display_name
- email
- role
- avatar_url
- created_at
```

```
authors
- id
- user_id
- name
- bio
- website_url
- status
```

```
contents
- id
- author_id
- title
- slug
- summary
- description
```

- category_id
- price_type
- price
- status
- thumbnail_url
- content_url
- created_at
- updated_at
- published_at

categories

- id
- name
- slug
- description

content_tags

- content_id
- tag

submissions

- id
- user_id
- title
- description
- file_url
- status
- admin_note
- created_at

favorites

- user_id
- content_id
- created_at

purchases

- id
- user_id
- content_id
- status
- amount
- created_at

最初から完璧にしなくていいけど、これを前提に画面を作ると後で強いです。

既存HTMLアプリはどうか

ここが現実的な落としどころです。

だこくん

勤怠データは個人情報に近いので、いきなりクラウド保存しない方がいいです。

最初は、

txt

- アプリ本体はマーケットに掲載
- データは利用者の端末内保存
- バックアップはJSON/Excel出力
- 将来的に企業アカウント版だけクラウド保存

が安全。

PDF編集

これはローカル処理のままでいいです。

txt

- PDFはアップロードしない
- ブラウザ内で処理
- 完了後にダウンロード

むしろセールスポイントになります。

座席管理

これはクラウド版にしてもいいです。

txt

- 教室ごとに座席表を保存
- 複数端末で共有
- スタッフごとにログイン

ただし、これはPhase 2以降。

面談支援ツール

これはクラウド保存に向いています。

txt

- 面談履歴
- 生徒情報

ただし、個人情報が入るので権限設計はかなり大事です。

僕ならこう決めます

LLLD Works Marketの基本方針はこれ。

txt

ポータル・アカウント・購入履歴
→ クラウド管理

各コンテンツの作業データ
→ 原則コンテンツごとに判断

PDF/画像/変換系
→ ローカル完結

勤怠/座席/面談/顧客管理系
→ 将来的にクラウド版を別商品化

投稿・審査・販売管理
→ クラウド管理

これが一番きれいです。

Codexに伝えるなら

次はこの指示を追加するといいです。

md

アカウント管理・クラウドDB化を見据えた設計方針

背景

LLLD Works Hubは、将来的に外部投稿型コンテンツプラットフォームへ拡張する予定です。
そのため、localStorageにポータル全体の重要データを保存するHTML単体アプリ設計から、認証・クラウドDB・ストレージを前提とした設計に移行します。

重要方針

- ポータル本体の重要データはlocalStorageに依存しない
- localStorageは一時保存・UI状態・無料ツールの一時作業データに限定する
- ユーザーアカウント、投稿者情報、コンテンツ一覧、購入履歴、公開ステータス、審査状況はクラウドDBで管理する前提にする
- まずはGitHub Pages + 静的HTML/CSS/JSでUIを作る

- データ取得部分は将来Supabase等に差し替えやすいように分離する

実装方針

現時点ではSupabase等への本接続はまだ必須ではありません。

ただし、以下のように設計してください。

- data/contents.json はDBの仮データとして扱う
- JS内で直接DOM生成とデータ定義を混ぜない
- `services/contentService.js` のような層を作り、将来APIやSupabaseに差し替えられるようにする
- `services/authService.js` のような層を作り、ログイン状態のモックを扱えるようにする
- `services/storageService.js` のような層を作り、画像・添付ファイル管理に備える
- UIは、未ログイン・ログイン済み・投稿者・管理者の状態を想定して作る

データ保存の原則

クラウド保存するもの

- ユーザー情報
- 投稿者情報
- コンテンツ情報
- カテゴリ
- タグ
- 投稿申請
- 公開/非公開ステータス
- お気に入り
- 購入履歴
- レビュー
- 管理者メモ

localStorageでもよいもの

- 最近見たコンテンツ
- UIの表示設定
- 入力途中の一時下書き
- ログイン不要ツールの一時作業データ

localStorageに保存しない方がよいもの

- アカウント情報
- 購入履歴
- 公開/非公開の権限情報
- 投稿審査情報
- 売上情報
- 個人情報を含む業務データ

今回のローカル検証版でやること

- JSONをDBの仮データとして扱う
- 将来Supabaseに置き換えられるファイル構成にする
- localStorage依存を増やさない
- アカウント制を想定した画面状態を作る
- 管理者・投稿者・一般ユーザーの表示差分をモックで確認できるようにする

将来想定

本格実装時は以下のような構成を想定します。

- Frontend: GitHub Pages または Cloudflare Pages
- Auth: Supabase Auth
- Database: Supabase Postgres
- Storage: Supabase Storage
- Payment: Stripe / Stripe Connect

まとめると

今の方向性は、

txt

HTML単体で全部動く社内ツール

から、

txt

静的フロント + クラウドDB + 認証 + コンテンツ管理

へ変える必要があります。

でも、いきなり全部クラウド化しなくていいです。

まずは、

txt

ローカルコピーでUIとJSON設計

↓

Supabase検証環境

↓

社内β

↓

外部投稿

↓

販売

が一番事故りません。

僕のおすすめは、**Supabase前提で設計だけ先に始める**です。

今すぐ接続しなくても、ファイル構成とJSの分け方をSupabaseに差し替えやすくしておく。

これが、今のLLLLD Works Hubからプラットフォームに昇格する一番きれいな道筋です。



xサーバーとかを契約して、そこで完結するとかってできるものなの？



ChatGPT gpt-5-5-thinking

できます。ただし「Xserverのどのサービスを使うか」で全然違います。静的サイトだけなら簡単、ログイン・DB・決済までやるなら構成をちゃんと選ぶ必要があります。少し最新仕様を確認して、現実的な選択肢に分けて整理します。

できる。

ただし、「Xserverのレンタルサーバー」で完結するのか、「XServer VPS」で完結するのかで別物です。

結論としては、LLLDの構想ならこうです。

いちばん現実的な答え

① Xserverレンタルサーバーだけで完結

これはできます。

ただし、向いているのは **WordPress型のコンテンツ販売・会員サイト** です。

できることは、

- 会員登録
- ログイン
- 投稿者管理
- コンテンツ一覧
- コンテンツ詳細ページ
- ダウンロード販売
- お問い合わせ
- ブログ/LP
- 管理画面
- メール
- WordPress運用

あたり。

XserverのレンタルサーバーはPHPが使える、MySQLも使えます。MySQLデータベース数・ユーザー数は無制限で、MySQL 5.7系やMariaDB 10.5/10.11系に対応しています。一方でPostgreSQLは利用不可です。(xserver.ne.jp)

なので、

WordPress + 会員プラグイン + ダウンロード販売プラグイン

みたいな構成なら、かなり現実的です。

ただし、だこくん、座席管理、面談支援、営業トーク支援みたいな**業務アプリ本体のデータ保存**まできれいにやるには、少し窮屈です。

② XServer VPSで完結

こっちはかなり自由です。

本格的にやるなら、こっちです。

できることは、

- Node.jsアプリ
- PHP/Laravel
- PostgreSQL
- Supabase
- Docker
- 独自API
- 認証
- DB
- ファイル保存
- 管理画面
- 決済連携
- アプリごとの保存データ管理

XServer VPSはOS・アプリイメージを選んでサーバーを作れるサービスで、Node.jsのアプリイメージも用意されています。(XServer VPS)

さらに、XServer VPSにはSupabaseアプリイメージもあり、VPS上にSupabase環境を構築でき、Supabase Studioも使えると説明されています。(XServer VPS)

つまり、

txt

XServer VPS

```
├── フロント画面
├── Supabase
│   ├── Auth
│   ├── Database
│   ├── Storage
│   └── 管理画面
├── 投稿管理
├── ユーザー管理
├── コンテンツ管理
└── 将来の決済連携
```

みたいに、**Xserver内でほぼ完結**できます。

これはかなりあり。

ただし、最初からVPSは少し重い

XServer VPSは自由度が高い反面、

- サーバー保守
- セキュリティ
- バックアップ
- SSL
- 障害対応
- Supabaseの更新
- DB管理
- 不正アクセス対策

を見ないといけません。

レンタルサーバーは楽。

VPSは強いけど、管理者が必要。

この違いです。

LLLDの今の段階ならおすすめは3択

案A: Xserverレンタルサーバー + WordPress

一番早く「外部投稿型コンテンツサイト」っぽくできます。

向いているもの：

- コンテンツ販売
- 資料販売
- テンプレート販売
- 投稿者紹介
- 会員限定ページ
- 無料/有料コンテンツ
- ブログ/LP
- 問い合わせ販売

ただし、業務アプリの中身までSaaS化するには弱い。

イメージはこれ。

txt

現場ツールマーケット
= WordPressで作る

各コンテンツ
= HTMLアプリ、PDF、Excel、ZIP、外部URLとして掲載

初期MVPならこれ、かなり堅いです。

案B: XServer VPS + Supabase

将来の本命。

向いているもの：

- アカウント管理
- 投稿者管理
- 管理者審査
- お気に入り
- 購入履歴
- コンテンツごとの権限管理
- アプリごとのクラウド保存
- 組織アカウント
- SaaS化

「ちゃんとプラットフォーム」にするならこれです。

ただし、開発・保守の難易度は上がります。

案C: GitHub Pages + Supabase公式クラウド

一番バランスがいい。

txt

GitHub Pages

= 画面・LP・フロント

Supabase公式クラウド

= ログイン・DB・ファイル保存

今の延長で作りやすいです。

サーバー管理をしなくていいので、Codex実装との相性もいいです。

個人的には、最初の開発はこれが一番おすすめ。

僕ならこう進める

いきなりXserverで全部完結させるより、段階を分けます。

Step 1: 今のGitHub版をローカルコピー

マーケット風UIを作る。

JSONで仮データ管理。

Step 2: Supabase公式クラウドで検証

アカウント、投稿者、コンテンツ、お気に入り、投稿申請を作る。

Step 3: 動きが見えたら判断

ここで分岐。

運用を楽にしたい

txt

GitHub Pages + Supabase公式クラウド

Xserver内で完結したい

txt

XServer VPS + Supabase自前構築

とりあえず販売サイト化したい

txt

Xserverレンタルサーバー + WordPress

LLLD構想に対する僕のおすすめ

本音で言うと、

最初のMVPは GitHub Pages + Supabase

これが一番いいです。

理由は、

- 今の資産を活かせる
- Codexで実装しやすい
- サーバー保守が少ない
- アカウント制にできる
- DB保存できる
- 将来移行しやすい

から。

その後、本格的に「自社で全部持ちたい」となったら、

XServer VPS + Supabase自前構築

に移す。

この順番が事故りにくいです。

Xserverだけで完結するなら最有力構成

「いや、できるだけXserver契約内で完結したい」という場合は、これです。

txt

```
XServer VPS
├── Nginx
├── Supabase
│   ├── Auth
│   ├── Postgres
│   ├── Storage
│   └── Studio
├── フロントエンド
├── 管理画面
└── コンテンツファイル
```

これなら、

- ログイン
- 投稿管理
- DB保存
- 画像保存
- ファイル保存
- 管理画面
- 権限管理

までできます。

ただ、これはもう「Webサービス運営」なので、ちゃんと管理する必要があります。

逆にレンタルサーバーだけだと厳しいもの

Xserverの普通のレンタルサーバーだけだと、以下はやりにくいです。

txt

- Supabase風の本格認証
- PostgreSQL前提の設計
- Node.js常駐アプリ
- WebSocket
- 複雑なAPI
- 投稿者ごとの細かい権限設計
- SaaSアプリごとのクラウド保存
- 管理者/投稿者/購入者の複雑な分岐

もちろんPHPで頑張ればできます。

でも、今後の拡張性を考えると、最初から無理やりレンタルサーバーだけに閉じる必要はないです。

まとめ

僕の判断はこれです。

txt

資料・テンプレ販売中心

→ Xserverレンタルサーバー + WordPressでOK

アカウント制のコンテンツマーケット

→ GitHub Pages + Supabaseが楽

自社完結の本格プラットフォーム

→ XServer VPS + Supabase

業務アプリのSaaS化まで狙う

→ VPS or 専用バックエンド必須

LLLLDの構想なら、最終的には **XServer VPS + Supabase** はかなり相性いいです。

ただ、最初からそこに行くより、まずは **GitHub Pages + Supabase公式クラウドで検証** が一番きれいだと思います。

2026-06-24 18:34:52

You Asked



Step 1: 今のGitHub版をローカルコピー

マーケット風UIを作る。

JSONで仮データ管理。

これが良さそうってことかな？